Universidade Estadual de Campinas
Instituto de Matemática, Estatística e Computação Científica

MS428 - Programação Linear

Projeto Computacional - Implementação do Algoritmo do Método Primal Simplex

Maria Eduarda Villéla Silva - RA 212974
Pabllo Ryan Teixeira Lima - RA 241219

Campinas - SP
Outubro de 2026

Sumário

1 Introdução **3**
1.1 O problema . . . 3
1.2 A ideia do Método Simplex . . . 3
1.3 O que este trabalho implementa . . . 3

2 O Método Simplex passo a passo **5**
2.1 Passo 1: solução básica . . . 5
2.2 Passo 2: vetor multiplicador simplex e custos relativos . . . 5
2.3 Passo 3: teste de otimalidade . . . 5
2.4 Passos 4 e 5: direção simplex, passo e variável que sai . . . 5
2.5 Passo 6: atualização da base . . . 6

3 Como o código está organizado **7**
3.1 Bloco 1: biblioteca e constantes . . . 7
3.2 Bloco 2: conferindo se a base inicial serve . . . 8
3.3 Bloco 3: o simplex . . . 8
3.4 Bloco 4: leitura dos dados digitados . . . 10
3.5 Bloco 5: exemplos prontos e programa principal . . . 11

4 Entrada e saída de dados **14**
4.1 Como executar . . . 14
4.2 Entrada (opção 0) . . . 14
4.3 Mensagens de erro . . . 14
4.4 Saída . . . 14

5 Exemplos **16**
5.1 Um problema com solução ótima (Opção 1) . . . 16
5.2 Um problema ilimitado (Opção 2) . . . 17
5.3 Quando $m = n$ (Opção 3) . . . 18
5.4 Exemplo com a Opção 0 (digitar os dados manualmente) . . . 18
5.5 Base inicial não factível . . . 20

6 Conclusão **22**

1 Introdução

1.1 O problema

A Programação Linear estuda a minimização (ou maximização) de uma função objetivo linear sujeita a restrições lineares de igualdade e/ou desigualdade. Todo problema de programação linear pode ser escrito na chamada **forma padrão**:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) = c^\mathsf{T} x \\ \text{sujeito a} \quad & Ax = b, \\ & x \geq 0, \end{aligned} \tag{1}$$

em que $A \in \mathbb{R}^{m \times n}$ é a matriz de coeficientes (de posto m), $b \in \mathbb{R}^m$ é o vetor de recursos e $c \in \mathbb{R}^n$ é o vetor de custos. Supõe-se $m < n$; o programa também aceita o caso $m = n$.

Como chegar à forma padrão: O programa espera o problema já na forma (1). Quando ele vem em outra forma, a conversão é feita antes de usar o programa:

- **Maximização:** maximizar $g(x)$ equivale a minimizar $-g(x)$; o valor ótimo do problema original é o oposto do valor encontrado.
- **Restrição $a^\mathsf{T} x \leq \beta$:** soma-se uma variável de *folga* $s \geq 0$, e fica $a^\mathsf{T} x + s = \beta$.
- **Restrição $a^\mathsf{T} x \geq \beta$:** subtrai-se uma variável de *excesso* $s \geq 0$, e fica $a^\mathsf{T} x - s = \beta$.
- **Termo independente negativo:** multiplica-se a restrição por -1.

As variáveis de folga e de excesso têm custo zero. Dentro da região factível, as folgas são positivas; na fronteira elas se anulam, o que indica restrições ativas, e em um vértice pelo menos $n - m$ variáveis são nulas.

1.2 A ideia do Método Simplex

O Método Primal Simplex, proposto por George B. Dantzig em 1947, procura a solução ótima entre os *vértices* da região factível $S = \{x \in \mathbb{R}^n : Ax = b,\ x \geq 0\}$. Para isso usa-se uma *partição básica* $A = [B\ N]$, em que $B \in \mathbb{R}^{m \times m}$ (matriz básica) reúne m colunas linearmente independentes de A e $N \in \mathbb{R}^{m \times (n-m)}$ (matriz não básica) reúne as $n - m$ restantes. Os vetores x e c são divididos de forma correspondente em $x = (x_B, x_N)$ e $c = (c_B, c_N)$. Fixando $x_N = 0$ obtém-se a *solução básica*

$$\hat{x}_B = B^{-1} b, \qquad \hat{x}_N = 0,$$

que é *factível* se $\hat{x}_B \geq 0$ e *não degenerada* se $\hat{x}_B > 0$.

Em resumo, a estratégia simplex é, a partir de uma solução básica factível, alterar apenas uma das variáveis não básicas (a que entra na base), o que define a *direção simplex* e o *tamanho do passo*.

1.3 O que este trabalho implementa

O algoritmo tem duas fases: a Fase I determina uma partição básica factível e a Fase II é a iteração simplex (Passos 1 a 6). **Este trabalho implementa apenas a Fase II.** No lugar da Fase I, o usuário informa uma *base inicial factível*, isto é, m índices de colunas de A tais que B seja inversível e $B^{-1} b \geq 0$. O programa confere essas condições antes de começar.

As entradas e saídas do programa são:

Entradas	m (restrições), n (variáveis), c (custos), b (recursos), A (coeficientes das restrições) e, por causa da Fase I ausente, uma base inicial factível.
Saídas	Se o problema for ilimitado, uma mensagem sobre isso. Se houver solução, o vetor x (solução ótima) e o valor ótimo $f(x)$.

O código foi escrito em Python na IDE Google Colab, com a biblioteca NumPy.

2 O Método Simplex passo a passo

2.1 Passo 1: solução básica

De $Ax = b$ com $A = [B\ N]$ obtém-se $Bx_B + Nx_N = b$, e portanto a solução geral

$$x_B = B^{-1}b - B^{-1}N\,x_N. \tag{2}$$

Com $x_N = 0$ tem-se a solução básica $\hat{x}_B = B^{-1}b$, cujo valor da função objetivo é $f(\hat{x}) = c_B^{\mathsf{T}}\hat{x}_B$. Em vez de inverter B, resolve-se o sistema linear $B\hat{x}_B = b$.

2.2 Passo 2: vetor multiplicador simplex e custos relativos

Escrevendo $f(x) = c^{\mathsf{T}}x$ com a partição básica e usando (2):

$$f(x) = c_B^{\mathsf{T}}x_B + c_N^{\mathsf{T}}x_N = c_B^{\mathsf{T}}B^{-1}b + \big(c_N^{\mathsf{T}} - c_B^{\mathsf{T}}B^{-1}N\big)x_N. \tag{3}$$

Define-se o **vetor multiplicador simplex**

$$\lambda^{\mathsf{T}} = c_B^{\mathsf{T}}B^{-1} \quad\Longleftrightarrow\quad B^{\mathsf{T}}\lambda = c_B \qquad \text{(Passo 2.1)} \tag{4}$$

e os **custos relativos** (ou reduzidos) das variáveis não básicas

$$\hat{c}_{N_j} = c_{N_j} - \lambda^{\mathsf{T}}a_{N_j}, \qquad j = 1,\ldots,n-m \qquad \text{(Passo 2.2)} \tag{5}$$

em que a_{N_j} é a coluna de A da variável x_{N_j}. Com isso, (3) fica

$$f(x) = f(\hat{x}) + \hat{c}_{N_1}x_{N_1} + \cdots + \hat{c}_{N_{n-m}}x_{N_{n-m}}, \tag{6}$$

isto é, $\hat{c}_{N_j}$ diz quanto f varia por unidade de aumento de x_{N_j}, mantendo as outras não básicas em zero. No Passo 2.3 escolhe-se, pela *regra de Dantzig*, a variável não básica de menor custo relativo, $\hat{c}_{N_k} = \min_j\{\hat{c}_{N_j}\}$.

2.3 Passo 3: teste de otimalidade

Condição de otimalidade: Seja $\hat{x}$ uma solução básica factível. Se $\hat{c}_{N_j} \geq 0$ para $j = 1,\ldots,n-m$, então $\hat{x}$ é solução ótima. Nesse caso, a iteração é interrompida.

2.4 Passos 4 e 5: direção simplex, passo e variável que sai

Se existe $\hat{c}_{N_k} < 0$, a variável x_{N_k} entra na base. Faz-se $x_{N_k} = \varepsilon \geq 0$, mantendo as outras não básicas nulas. Para que $Ax = b$ continue valendo, as variáveis básicas mudam para

$$x_B(\varepsilon) = \hat{x}_B - \varepsilon\, y, \qquad y = B^{-1}a_{N_k} \quad (\text{resolve-se } By = a_{N_k}), \tag{7}$$

O vetor y é a **direção simplex** (Passo 4). Como é preciso manter $x_{B_i} = \hat{x}_{B_i} - \varepsilon y_i \geq 0$ para todo i, há dois casos (Passo 5):

- $y \leq 0$**:** nenhuma variável básica diminui, então $x(\varepsilon)$ é factível para todo $\varepsilon \geq 0$ e $f \to -\infty$: o problema é **ilimitado**.
- **Existe** $y_i > 0$**:** só as variáveis com $y_i > 0$ diminuem, e o **tamanho do passo** máximo é

$$\hat{\varepsilon} = \frac{\hat{x}_{B_\ell}}{y_\ell} = \min\left\{\frac{\hat{x}_{B_i}}{y_i} \;:\; y_i > 0\right\}. \tag{8}$$

2.5 Passo 6: atualização da base

A variável x_{B_ℓ} anula-se e sai da base; x_{N_k} passa a valer $\hat{\varepsilon}$ e entra. Troca-se a ℓ-ésima coluna de B pela k-ésima coluna de N ($B_\ell \leftrightarrow N_k$). A nova matriz B' continua inversível: $B^{-1}B'$ é a identidade com a coluna ℓ trocada por y, e seu determinante é $y_\ell > 0$. Assim $A = [B'\ N']$ é uma nova partição básica, e o novo valor da função objetivo é $f(\hat{x}) + \hat{\varepsilon}\,\hat{c}_{N_k} \leq f(\hat{x})$.

3 Como o código está organizado

O arquivo `primal_simplex.py` tem cinco blocos: constantes, verificação da base inicial, o algoritmo da Fase II, leitura dos dados e, por fim, exemplos e programa principal. A seguir, cada bloco é mostrado e explicado.

Convenções: Internamente, o Python numera as posições a partir de 0; para o usuário (e na saída) as variáveis são $x_1, \ldots, x_n$. Por isso há conversões ± 1 na leitura da base e na impressão. Os vetores `base` e `nao_base` guardam os índices $(B_1, \ldots, B_m)$ e $(N_1, \ldots, N_{n-m})$. A *ordem* desses vetores importa: `xB`, `y` e `razoes` seguem a ordem de `base`, e `cr` segue a ordem de `nao_base`. Além disso, `cNk` e `p` são *posições* dentro de `nao_base` e `base`, enquanto `nao_base[cNk]` e `base[p]` são os *índices* das variáveis que entram e saem.

Algoritmo	Código	Observação
Partição inicial	`base = list(base); nao_base = [j for j in range(n) if j not in base]`	Substitui a Fase I.
Passo 1	`xB = np.linalg.solve(B, b); fx = cB @ xB`	$B\hat{x}_B = b$ e $c_B^{\mathsf{T}}\hat{x}_B$.
Passo 2.1	`lamb = np.linalg.solve(B.T, cB)`	$B^{\mathsf{T}}\lambda = c_B$.
Passo 2.2	`cr = cN - N.T @ lamb`	Todos os $\hat{c}_{N_j}$ de uma vez.
Passo 2.3	`cNk = np.argmin(cr)`	Posição do menor custo relativo.
Passo 3	`if cr[cNk] >= -tolerancia: return "ótimo", ...`	O `return` faz o papel de `PARE = VERDADE`.
Passo 4	`y = np.linalg.solve(B, A[:, entra_na_base])`	$By = a_{N_k}$.
Passo 5	`if np.all(y <= tolerancia): return "ilimitado", ...; razoes, pos = y > 0, p = np.argmin(razoes)`	Ilimitado se $y \leq 0$; senão, razão mínima só onde $y_i > 0$.
Passo 6	`base[p], nao_base[cNk] = nao_base[cNk], base[p]`	Troca $B_\ell \leftrightarrow N_k$.

3.1 Bloco 1: biblioteca e constantes

```
1 import numpy as np
2
3 tolerancia = 1e-9 #define o menor valor que o programa considera diferente de 0
4 max_iter = 1000 #número máximo de iterações para evitar ciclagem
```

Listing 1: Importação do NumPy e constantes globais.

- `import numpy as np`: traz a biblioteca NumPy, usada para vetores e matrizes e para resolver sistemas lineares.
- `tolerancia = 1e-9`: contas em ponto flutuante têm pequenos erros de arredondamento, e algo que deveria ser 0 pode sair como $-3 \cdot 10^{-17}$. Para que esse ruído não seja confundido

com um valor negativo, os testes comparam com $\pm 10^{-9}$: factibilidade da base ($x_B \geq -\delta$), otimalidade ($\hat{c}_N \geq -\delta$) e ilimitação ($y \leq \delta$), com $\delta = 10^{-9}$. No cálculo das razões do Passo 5 o código usa a comparação `y > 0`, sem tolerância.

- `max_iter = 1000`: limite de iterações. Esse limite é só uma trava de segurança para o código não rodar infinitamente e, se for atingido, não quer dizer que o problema seja ilimitado.

3.2 Bloco 2: conferindo se a base inicial serve

Como a Fase I não foi implementada, o programa confere, antes de começar, se a base informada pode ser usada como ponto de partida.

```
def verifica_base_factivel(A, b, base):
  #verifica se a base informada pelo usuário é válida e factível
  m, n = A.shape
  if len(base) != m or len(set(base)) != m:
    raise ValueError(f"A base informada deve ter {m} índices distintos.")
  if any(j < 0 or j >= n for j in base):
    raise ValueError(f"Os índices da base devem estar entre 1 e {n}.")

  B = A[:, base]
  if abs(np.linalg.det(B)) < tolerancia:
    raise ValueError("A matriz informada não é invertível.")

  xB = np.linalg.solve(B, b)
  if np.any(xB < -tolerancia):
    raise ValueError(f"Base não factível: xB = {xB}")
```

Listing 2: Verificação de validade e factibilidade da base inicial.

1. A lista `base` precisa ter exatamente m índices, sem repetição (`len(set(base)) != m`).
2. Cada índice deve estar entre 0 e $n-1$ (a numeração interna do Python; o usuário digita de 1 a n).
3. B **inversível:** extrai $B = A[:, \text{base}]$ e testa se $|\det B| \geq 10^{-9}$. Caso contrário, as colunas escolhidas são dependentes e não formam uma base.
4. **Factibilidade:** resolve $Bx_B = b$ e confere se todas as componentes satisfazem $x_B \geq -10^{-9}$. Se alguma for negativa, a base é infactível.

Qualquer falha interrompe o programa com uma mensagem específica.

3.3 Bloco 3: o simplex

A função `fase2_simplex` é a parte principal do programa. Ela recebe A, b, c e a base inicial e devolve `(status, x, fx, it)`, em que `status` é `"ótimo"` ou `"ilimitado"`. A explicação está dividida em duas partes: Passos 1 a 3 (Código 3) e Passos 4 a 6 (Código 4).

3A: Passos 1 a 3

```python
def fase2_simplex(A, b, c, base):
  #fase2_simplex: retorna (status, x, fx, it); status: ótimo ou ilimitado
  m, n = A.shape #m: linhas e n: colunas
  base = list(base)
  nao_base = [j for j in range(n) if j not in base] #lista c/ as colunas não base
  it = 0

  while it < max_iter:
    B = A[:, base] #B = todas as linhas de A, colunas da base
    N = A[:, nao_base] #N = todas as linhas de A, colunas da não base (fora da base)
    cB = c[base] #coeficientes das variáveis básicas
    cN = c[nao_base] #coeficientes das variáveis não básicas

    #PASSO 1: solução básica
    xB = np.linalg.solve(B, b)
    fx = cB @ xB
    x = np.zeros(n)
    x[base] = xB

    #caso m == n: não tem variável não básica
    if len(nao_base) == 0:
      return "ótimo", x, fx, it

    #PASSO 2: custos relativos (das variáveis não básicas)
    lamb = np.linalg.solve(B.T, cB) #passo 2.1 - vetor multiplicador simplex
    cr = cN - N.T @ lamb #passo 2.2 - custos relativos
    cNk = np.argmin(cr) #passo 2.3 - escolha da variável a entrar na base

    print("-" * 60) #linha para separar as iterações
    print(f"Iteração {it}")
    print(f"Base: ", [i + 1 for i in base])
    print(f"Não base: ", [j + 1 for j in nao_base])
    print(f"xB: ", np.round(xB, 6))
    print(f"fx: ", round(fx, 6))
    print(f"λ: ", np.round(lamb, 6))
    print(f"Custos relativos: ", np.round(cr, 6))

    #PASSO 3: teste de otimalidade
    if cr[cNk] >= -tolerancia:
      print("custos relativos >= 0, então a solução é ótima")
```

Listing 3: Partição inicial e Passos 1, 2 e 3.

- **Partição e matrizes:** a partir de `base` monta-se o complemento `nao_base` e, a cada iteração, extraem-se B, N, c_B e c_N com os índices atuais. Isso é refeito a cada volta do `while` porque a base muda.
- **Passo 1:** em vez de inverter B, resolve $Bx_B = b$ com `np.linalg.solve`; calcula $f = c_B^{\mathsf{T}} x_B$ (`cB @ xB`) e monta o vetor completo $x \in \mathbb{R}^n$, com as não básicas iguais a zero.
- **Caso $m = n$:** se `nao_base` é vazia, não há o que melhorar: o sistema $Bx_B = b$ tem solução única, que é devolvida como ótima. Sem esse teste, o `np.argmin` do Passo 2.3 receberia um

vetor vazio.

- **Passo 2:** resolve o sistema transposto $B^{\mathsf{T}}\lambda = c_B$ (`lamb`), calcula todos os custos relativos $\hat{c}_N = c_N - N^{\mathsf{T}}\lambda$ (`cr`) e guarda em `cNk` a posição do menor deles (regra de Dantzig).
- **Impressões:** o bloco de `print` mostra o estado da iteração.
- **Passo 3:** se o menor custo relativo é $\geq -10^{-9}$, nenhuma variável não básica melhora f e a solução atual é ótima; o `return` encerra a função.

3B: Passos 4 a 6

```
43      #PASSO 4: direção simplex
44      entra_na_base = nao_base[cNk]
45      y = np.linalg.solve(B, A[:, entra_na_base])
46      print(f"x{entra_na_base + 1} entra na base | y:", np.round(y, 6))
47
48      if np.all(y <= tolerancia):
49        print("y <= 0, então não existe solução ótima finita. Problema ilimitado."
        )
50        return "ilimitado", None, None, it
51
52      #PASSO 5: determinar passo e variável a sair da base
53      razoes = np.full(m, np.inf) #inf: infinito onde y <= 0 (não limita)
54      pos = y > 0
55      razoes[pos] = xB[pos] / y[pos]
56      p = np.argmin(razoes) #posição da variável que sai da base
57      print(f"x{base[p] + 1} sai da base | passo = {razoes[p]:.6f}")
58
59      #PASSO 6: atualização (nova partição básica)
60      base[p], nao_base[cNk] = nao_base[cNk], base[p]
61      it = it + 1
62
63  raise RuntimeError("Número máximo de iterações atingido.")
```

Listing 4: Passos 4, 5 e 6.

- **Passo 4:** `entra_na_base = nao_base[cNk]` é o índice da variável que entra. A direção simplex y vem de $By = a_{N_k}$, em que a_{N_k} é a coluna `A[:, entra_na_base]`. O vetor y diz quanto cada variável básica diminui por unidade de aumento da que entra ($x_B(\varepsilon) = \hat{x}_B - \varepsilon y$).
- **Ilimitado:** se todas as componentes de y são $\leq 10^{-9}$, nenhuma variável básica diminui; ε cresce sem limite e $f \to -\infty$.
- **Passo 5:** `razoes` começa com $+\infty$ em todas as posições; `pos = y > 0` marca as linhas com $y_i > 0$, e só nelas calcula-se $\hat{x}_{B_i}/y_i$. O menor valor está na posição `p`, que indica a variável que sai (`base[p]`), e o valor $\hat{\varepsilon}$ é impresso como "passo". Em empates, `np.argmin` escolhe a primeira posição.
- **Passo 6:** a troca `base[p], nao_base[cNk] = nao_base[cNk], base[p]` põe a variável que entra no lugar da que sai, e `it` conta mais uma iteração. Se o laço terminar sem `return`, o limite `max_iter` foi atingido e o programa levanta um erro.

3.4 Bloco 4: leitura dos dados digitados

```python
def le_vetor(texto, tamanho):
  #lê "tamanho" números separados por espaço (aceita vírgula decimal)
  while True:
    partes = input(texto).replace(",", ".").split()
    try:
      vet = np.array([float(p) for p in partes])
    except ValueError:
      print("  Digite apenas números.")
      continue
    if len(vet) != tamanho:
      print(f"  Esperado {tamanho} valores, recebido {len(vet)}.")
      continue
    return vet

#----------------------------------------

def entrada():
  m = int(input("Digite o número de restrições: "))
  n = int(input("Digite o número de variáveis: "))
  if m > n:
    raise ValueError("Precisamos de restrições <= variáveis.")

  c = le_vetor(f"c ({n} valores): ", n)
  b = le_vetor(f"b ({m} valores): ", m)
  A = np.zeros((m, n))
  for i in range(m):
    A[i, :] = le_vetor(f"Linha {i + 1} de A ({n} valores): ", n)

  base = le_vetor(f"Base inicial ({m} índices de 1 a {n}): ", m)
  base = [int(j) - 1 for j in base] #1-based -> 0-based

  return A, b, c, base
```

Listing 5: Leitura de vetores e entrada manual.

- `le_vetor(texto, tamanho)`: lê uma linha, troca vírgula por ponto (`.replace(",", ".")`) para aceitar a vírgula decimal, separa nos espaços (`.split()`) e converte cada pedaço para número. O `try/except` pega o erro de conversão quando o usuário digita algo que não é número, e o programa repete a pergunta. A pergunta também é repetida se a quantidade de valores não for a esperada.

- `entrada()`: pede m e n (exigindo $m \leq n$), depois c, b, as m linhas de A (cada linha preenchida com `A[i, :] = ...`) e a base inicial. Os índices da base são digitados de 1 a n e convertidos para $0, \ldots, n-1$ por `[int(j) - 1 for j in base]`.

3.5 Bloco 5: exemplos prontos e programa principal

```python
def exemplo(numero):
  #1: ótimo (x = (3,1,0,0,0), f = -11) | 2: ilimitado | 3: caso m == n
  if numero == 1:
    A = [[1, 1, 1, 0, 0],
         [1, 3, 0, 1, 0],
         [1, 0, 0, 0, 1]]
    b = [4, 6, 3]
    c = [-3, -2, 0, 0, 0]
    base = [2, 4, 3]
  elif numero == 2:
    A = [[1, -1, 1, 0],
         [-1, 1, 0, 1]]
    b =[1, 1]
    c = [-1, -1, 0, 0]
    base = [2, 3]
  else:
    A = [[1, 0],
         [0, 1]]
    b = [1, 2]
    c = [1, 1]
    base = [0, 1]
  return (np.array(A, dtype = float),
          np.array(b, dtype = float),
          np.array(c, dtype = float),
          base)

#----------------------------------------

def main():
  print("Método Primal Simplex (Fase II)")
  print(" " * 60)
  print("Escolha uma das opções abaixo:")
  print("0) Digitar os dados manualmente")
  print("1) Usar exemplo 1 (ótimo)")
  print("2) Usar exemplo 2 (ilimitado)")
  print("3) Usar exemplo 3 (caso m == n)")
  opcao = input("Opção: ").strip()

  if opcao == "0":
    A, b, c, base = entrada()
  else:
    A, b, c, base = exemplo(int(opcao))

  verifica_base_factivel(A, b, base)
  status, x, fx, it = fase2_simplex(A, b, c, base)

  print(" " * 60)
  print("Iterações: ", it)
  if status == "ilimitado":
    print("Resultado: problema ilimitado.")
  else:
    print("Resultado: solução ótima.")
    for i in range(len(x)):
      print(f"  x{i + 1} = {x[i]:.6f}")
    print(f"  f(x) = {fx:.6f}")


if __name__ == "__main__":
  main()
```

Listing 6: Banco de exemplos internos e função `main`.

- `exemplo(numero)`: devolve três problemas prontos, que cobrem os caminhos do algoritmo: solução ótima, problema ilimitado e o caso $m = n$. As listas são convertidas para `np.array`

do tipo `float`.

- `main()`: mostra o menu, obtém os dados (`entrada()` para a opção `0`, `exemplo()` para as opções `1` a `3`), chama `verifica_base_factivel`, depois `fase2_simplex`, e imprime o resultado final com seis casas decimais.

- `if __name__ == "__main__"`: é o que inicia o programa quando o arquivo é executado diretamente. Se o arquivo for importado por outro código, as funções ficam disponíveis, mas `main()` não roda sozinha.

4 Entrada e saída de dados

4.1 Como executar

O programa roda em um terminal (`python primal_simplex.py`) ou em uma célula do Google Colab. Ao iniciar, aparece um menu:

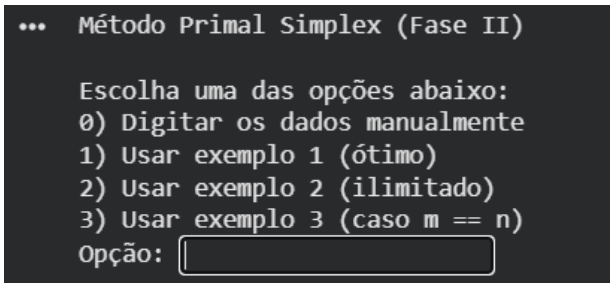


Figura 1: Menu inicial do programa Método Primal Simplex (Fase II).

4.2 Entrada (opção 0)

Os dados são pedidos nesta ordem:

1. o número de restrições m e o número de variáveis n (inteiros, com $m \leq n$);
2. o vetor de custos c (n valores);
3. o vetor de recursos b (m valores);
4. a matriz A, *uma linha por vez* (m linhas com n valores);
5. a base inicial factível: m índices de 1 a n (por exemplo, `3 4 5` para x_3, x_4, x_5).

Cada vetor ou linha é digitado em uma só linha, com os números **separados por espaço**. A vírgula decimal é aceita. Se a quantidade de valores estiver errada ou aparecer algo que não é número, o programa avisa e pergunta de novo.

4.3 Mensagens de erro

Situação	Mensagem
$m > n$	`Precisamos de restrições <= variáveis.`
Quantidade errada de índices na base, ou índices repetidos	`A base informada deve ter m índices distintos.`
Índice fora de $1, \ldots, n$	`Os índices da base devem estar entre 1 e n.`
B singular	`A matriz informada não é invertível.`
$B^{-1}b$ com componente negativa	`Base não factível:  xB = [...]`
Limite de iterações atingido	`Número máximo de iterações atingido.`

4.4 Saída

Passo a passo. A cada iteração o programa imprime:

Linha impressa	**Significado**
`Iteração k`	Número da iteração, começando em 0.
`Base`	Índices das variáveis básicas ($B_1, \ldots, B_m$), na ordem das colunas de B.
`Não base`	Índices das não básicas ($N_1, \ldots, N_{n-m}$), na ordem das colunas de N.
`xB`	Valores das básicas ($\hat{x}_B = B^{-1}b$), na mesma ordem de `Base`.
`fx`	Valor atual da função objetivo, $c_B^{\mathsf{T}}\hat{x}_B$.
λ	Vetor multiplicador simplex (um valor por restrição).
`Custos relativos`	$\hat{c}_N$, na mesma ordem de `Não base`.
`x`j `entra na base | y:`	Variável escolhida para entrar e a direção simplex y (na ordem de `Base`).
`x`j `sai da base | passo`	Variável que sai e o tamanho do passo $\hat{\varepsilon}$.

Quando o algoritmo para, uma frase informa o motivo: custos relativos ≥ 0 (solução ótima) ou $y \leq 0$ (problema ilimitado). Em cada iteração a primeira linha é um traço para separar os blocos.

Ao final aparecem o número de iterações e:

- se o problema é ilimitado: `Resultado: problema ilimitado.`;
- se há solução ótima: o vetor x completo ($x_1, \ldots, x_n$, com as não básicas iguais a zero) e o valor ótimo $f(x)$, com seis casas decimais.

O valor impresso em `Iterações` conta as *trocas de base*. Como a última passagem pelo laço só confirma a otimalidade, aparecem sempre um bloco a mais do que o número de iterações. Por exemplo, `Iterações: 2` corresponde a três blocos impressos (`Iteração 0`, `1` e `2`).

5 Exemplos

Os problemas abaixo estão na forma padrão (ou foram convertidos para ela) e as saídas mostradas foram geradas pelo próprio programa.

5.1 Um problema com solução ótima (Opção 1)

O Exemplo 1 vem da maximização de $3x_1 + 2x_2$ sujeita a $x_1 + x_2 \leq 4$, $x_1 + 3x_2 \leq 6$ e $x_1 \leq 3$, com $x_1, x_2 \geq 0$. Trocando por minimização e somando as folgas $x_3, x_4, x_5 \geq 0$, obtém-se $m = 3$, $n = 5$:

$$\min\ f(x) = -3x_1 - 2x_2, \qquad A = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 3 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ 6 \\ 3 \end{bmatrix}, \quad x \geq 0.$$

A base inicial é (x_3, x_5, x_4) (índices `[2, 4, 3]` no Python), isto é, as três folgas, apenas em outra ordem. Ela corresponde à origem $(x_1, x_2) = (0, 0)$, com $x_3 = 4$, $x_5 = 3$ e $x_4 = 6$.

```
Método Primal Simplex (Fase II)

Escolha uma das opções abaixo:
0) Digitar os dados manualmente
1) Usar exemplo 1 (ótimo)
2) Usar exemplo 2 (ilimitado)
3) Usar exemplo 3 (caso m == n)
Opção: 1
------------------------------------------------------------
Iteração 0
Base:  [3, 5, 4]
Não base:  [1, 2]
xB:  [4. 3. 6.]
fx:  0.0
λ:  [0. 0. 0.]
Custos relativos:  [-3. -2.]
x1 entra na base | y: [1. 1. 1.]
x5 sai da base | passo = 3.000000
------------------------------------------------------------
Iteração 1
Base:  [3, 1, 4]
Não base:  [5, 2]
xB:  [1. 3. 3.]
fx:  -9.0
λ:  [ 0.  0. -3.]
Custos relativos:  [ 3. -2.]
x2 entra na base | y: [1. 0. 3.]
x3 sai da base | passo = 1.000000
------------------------------------------------------------
Iteração 2
Base:  [2, 1, 4]
Não base:  [5, 3]
xB:  [ 1.  3. -0.]
fx:  -11.0
λ:  [-2. -0. -1.]
Custos relativos:  [1. 2.]
custos relativos >= 0, então a solução é ótima

Iterações:  2
Resultado: solução ótima.
  x1 = 3.000000
  x2 = 1.000000
  x3 = 0.000000
  x4 = -0.000000
  x5 = 0.000000
  f(x) = -11.000000
```

Figura 2: Saída do programa para a Opção 1.

O que acontece em cada iteração.

- **Iteração 0:** os custos relativos de x_1 e x_2 são -3 e -2, e x_1 entra. As razões são $4/1$, $3/1$ e $6/1$ para x_3, x_5 e x_4, então sai x_5 (o limite é a restrição $x_1 \leq 3$), com passo $\hat{\varepsilon} = 3$. A função cai de 0 para $0 + 3 \cdot (-3) = -9$.
- **Iteração 1:** entra x_2 ($\hat{c}_2 = -2$). As razões de x_3 e de x_4 *empatam* em 1 ($1/1 = 3/3$); a linha de x_1 tem $y = 0$ e não conta. `np.argmin` escolhe a primeira posição da base, e sai x_3.
- **Iteração 2:** $\hat{c}_N = (1, 2) \geq 0$, e a solução é ótima. Aqui $x_4 = 0$ permanece na base: a solução é *degenerada*.

5.2 Um problema ilimitado (Opção 2)

O Exemplo 2 ($m = 2$, $n = 4$) tem uma região factível aberta na direção em que a função objetivo diminui:

$$\min\ f(x) = -x_1 - x_2, \qquad A = \begin{bmatrix} 1 & -1 & 1 & 0 \\ -1 & 1 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad x \geq 0.$$

No plano (x_1, x_2), a região é a faixa $-1 \leq x_1 - x_2 \leq 1$ no primeiro quadrante, onde x_1 e x_2 podem crescer juntos indefinidamente. A base inicial é (x_3, x_4).

```
Método Primal Simplex (Fase II)

Escolha uma das opções abaixo:
0) Digitar os dados manualmente
1) Usar exemplo 1 (ótimo)
2) Usar exemplo 2 (ilimitado)
3) Usar exemplo 3 (caso m == n)
Opção: 2
------------------------------------------------------------
Iteração 0
Base:  [3, 4]
Não base:  [1, 2]
xB:  [1. 1.]
fx:  0.0
λ:  [0. 0.]
Custos relativos:  [-1. -1.]
x1 entra na base | y: [ 1. -1.]
x3 sai da base | passo = 1.000000
------------------------------------------------------------
Iteração 1
Base:  [1, 4]
Não base:  [3, 2]
xB:  [1. 2.]
fx:  -1.0
λ:  [-1.  0.]
Custos relativos:  [ 1. -2.]
x2 entra na base | y: [-1.  0.]
y <= 0, então não existe solução ótima finita. Problema ilimitado.

Iterações:  1
Resultado: problema ilimitado.
```

Figura 3: Saída do programa para a Opção 2.

- **Iteração 0:** x_1 entra, com $y = (1, -1)$. Só a linha de x_3 limita, com passo 1, e sai x_3. O ponto passa a ser $(x_1, x_2, x_3, x_4) = (1, 0, 0, 2)$.
- **Iteração 1:** x_2 entra ($\hat{c}_2 = -2$) e $y = (-1, 0) \leq 0$. Ao aumentar $x_2 = \varepsilon$, as variáveis básicas ficam $x_1 = 1 + \varepsilon$ e $x_4 = 2$: nenhuma diminui, então ε cresce sem limite e $f = -1 - 2\varepsilon \to -\infty$. O programa detecta $y \leq 0$ e informa que o problema é ilimitado.

5.3 Quando $m = n$ (Opção 3)

O Exemplo 3 ($m = n = 2$) tem o sistema quadrado $I_2 x = (1, 2)^\mathsf{T}$, $c = (1, 1)^\mathsf{T}$ e base (x_1, x_2). Como $B = A$, não há variáveis não básicas e a única solução do sistema é $x = (1, 2)$. O programa a devolve na iteração 0, com custo $f(x) = 3$:

```
•••  Método Primal Simplex (Fase II)

     Escolha uma das opções abaixo:
     0) Digitar os dados manualmente
     1) Usar exemplo 1 (ótimo)
     2) Usar exemplo 2 (ilimitado)
     3) Usar exemplo 3 (caso m == n)
     Opção: 3

     Iterações:  0
     Resultado: solução ótima.
       x1 = 1.000000
       x2 = 2.000000
       f(x) = 3.000000
```

Figura 4: Saída do programa para a Opção 3.

5.4 Exemplo com a Opção 0 (digitar os dados manualmente)

Tome o seguinte problema de programação linear:

$$\min \; f(x) = -2x_1 - x_2 \quad \text{s.a} \quad \begin{cases} x_1 + x_2 + x_3 = 6 \\ x_1 - x_2 + x_4 = 4 \\ 3x_1 + x_2 - x_5 = 3 \end{cases} \qquad x \geq 0,$$

Aqui x_5 é uma variável de excesso (coeficiente -1), por isso as variáveis de folga não formam uma base factível e a base inicial precisa ser escolhida.

Partição (x_1, x_2, x_5): O programa mostra que ela já é ótima, sem nenhuma troca de base:

```
Método Primal Simplex (Fase II)

Escolha uma das opções abaixo:
0) Digitar os dados manualmente
1) Usar exemplo 1 (ótimo)
2) Usar exemplo 2 (ilimitado)
3) Usar exemplo 3 (caso m == n)
Opção: 0
Digite o número de restrições: 3
Digite o número de variáveis: 5
c (5 valores): -2 -1 0 0 0
b (3 valores): 6 4 3
Linha 1 de A (5 valores): 1 1 1 0 0
Linha 2 de A (5 valores): 1 -1 0 1 0
Linha 3 de A (5 valores): 3 1 0 0 -1
Base inicial (3 índices de 1 a 5): 1 2 5
--------------------------------------------------
Iteração 0
Base:  [1, 2, 5]
Não base:  [3, 4]
xB:  [ 5.  1. 13.]
fx:  -11.0
λ:  [-1.5 -0.5 -0. ]
Custos relativos:  [1.5 0.5]
custos relativos >= 0, então a solução é ótima

Iterações:  0
Resultado: solução ótima.
  x1 = 5.000000
  x2 = 1.000000
  x3 = 0.000000
  x4 = 0.000000
  x5 = 13.000000
  f(x) = -11.000000
```

Figura 5: Base inicial $(1, 2, 5)$

Partição (x_1, x_3, x_5)**:** Aqui o programa faz uma troca de base e chega à primeira partição:

```
Método Primal Simplex (Fase II)

Escolha uma das opções abaixo:
0) Digitar os dados manualmente
1) Usar exemplo 1 (ótimo)
2) Usar exemplo 2 (ilimitado)
3) Usar exemplo 3 (caso m == n)
Opção: 0
Digite o número de restrições: 3
Digite o número de variáveis: 5
c (5 valores): -2 -1 0 0 0
b (3 valores): 6 4 3
Linha 1 de A (5 valores): 1 1 1 0 0
Linha 2 de A (5 valores): 1 -1 0 1 0
Linha 3 de A (5 valores): 3 1 0 0 -1
Base inicial (3 índices de 1 a 5): 1 3 5
------------------------------------------------
Iteração 0
Base:  [1, 3, 5]
Não base:  [2, 4]
xB:  [4. 2. 9.]
fx:  -8.0
λ:  [ 0. -2. -0.]
Custos relativos:  [-3.  2.]
x2 entra na base | y: [-1.  2. -4.]
x3 sai da base | passo = 1.000000
------------------------------------------------
Iteração 1
Base:  [1, 2, 5]
Não base:  [3, 4]
xB:  [ 5.  1. 13.]
fx:  -11.0
λ:  [-1.5 -0.5 -0. ]
Custos relativos:  [1.5 0.5]
custos relativos >= 0, então a solução é ótima

Iterações:  1
Resultado: solução ótima.
  x1 = 5.000000
  x2 = 1.000000
  x3 = 0.000000
  x4 = 0.000000
  x5 = 13.000000
  f(x) = -11.000000
```

Figura 6: Base inicial $(1, 3, 5)$

Na primeira iteração, o custo relativo $\hat{c}_2 = -3 < 0$ faz x_2 entrar. Resolvendo $By = a_2$ obtém-se $y = (-1, 2, -4)$; só a componente de x_3 é positiva, então a razão mínima é $2/2 = 1$ e x_3 sai. As variáveis básicas passam a $x_1 = 4 - 1 \cdot (-1) = 5$, $x_2 = 1$ e $x_5 = 9 - 1 \cdot (-4) = 13$, e $f = -8 + 1 \cdot (-3) = -11$. A solução ótima é $x^* = (5, 1, 0, 0, 13)$ com $f(x^*) = -11$.

5.5 Base inicial não factível

Se o usuário informar, no problema do Exemplo 1, a base (x_2, x_4, x_5), a solução básica é $\hat{x}_B = (4, -6, 3)$, com componente negativa. O programa interrompe a execução antes de iniciar o simplex, com a mensagem abaixo (a última linha do erro):

```
Escolha uma das opções abaixo:
0) Digitar os dados manualmente
1) Usar exemplo 1 (ótimo)
2) Usar exemplo 2 (ilimitado)
3) Usar exemplo 3 (caso m == n)
Opção: 0
Digite o número de restrições: 3
Digite o número de variáveis: 5
c (5 valores): -3 -2 0 0 0
b (3 valores): 4 6 3
Linha 1 de A (5 valores): 1 1 1 0 0
Linha 2 de A (5 valores): 1 3 0 1 0
Linha 3 de A (5 valores): 1 0 0 0 1
Base inicial (3 índices de 1 a 5): 2 4 5
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1293/3572557374.py in <cell line: 0>()
    181
    182 if __name__ == "__main__":
--> 183   main()

                                 1 frames
/tmp/ipykernel_1293/3572557374.py in verifica_base_factivel(A, b, base)
     84   xB = np.linalg.solve(B, b)
     85   if np.any(xB < -tolerancia):
---> 86     raise ValueError(f"Base não factível: xB = {xB}")
     87
     88 #----------------------------------------

ValueError: Base não factível: xB = [ 4. -6.  3.]
```

Figura 7: Base não factível informada pelo usuário

6 Conclusão

Neste trabalho, foi realizada a implementação em Python da Fase II do Método Primal Simplex, seguindo seis passos: cálculo da solução básica, do vetor multiplicador simplex e dos custos relativos, teste de otimalidade, direção simplex, razão mínima e troca de base. O programa recebe m, n, c, b, A e uma base inicial factível; devolve o vetor ótimo x e o valor $f(x)$, ou informa que o problema é ilimitado.

Os testes cobriram os caminhos do algoritmo. O Exemplo 1 terminou em uma solução ótima degenerada, com empate na razão mínima. O Exemplo 2 foi detectado como ilimitado, e o Exemplo 3 mostrou o caso $m = n$. O Exemplo da Opção 0 recebeu uma entrada manual e foi testado com duas partições diferentes.

Referências

[1] POLDI, K. *O algoritmo primal simplex*. Material de aula (Aula 9, 10 set. 2026), disciplina MS428 – Programação Linear, Google Classroom.

[2] POLDI, K. *Método Simplex (slides – Aulas 7, 8 e 9)*. Material de aula da disciplina MS428 – Programação Linear, 14 set. 2026.

[3] BAZARAA, M. S.; JARVIS, J. J.; SHERALI, H. D. *Linear Programming and Network Flows*. 4. ed. Wiley, 2010.